

CSE 314 - Software Engineering Sessional

Practice Set on Behavioral Design Patterns

Below are 10 practice problems, two for each of the five behavioral patterns covered in the course: **State**, **Strategy**, **Observer**, **Mediator**, and **Template Method**. Following the style of the reference exam papers, some problems intentionally combine a second pattern on top of the primary one -- read each problem carefully to identify every pattern that needs to be applied. Each problem is followed by a short list of expectations, similar to the reference papers, describing what your program must demonstrate.

Subsection: S1 Time: 25 minutes

Pattern focus: State Pattern

A software company sells a **Smart Lock** for apartment doors. The lock can be in one of four states: **Locked**, **Unlocked**, **Jammed**, and **LowBattery**. The lock supports four operations: **enterPin(pin)**, **lockButton()**, **forceUnlock(masterKey)**, and **drainBattery(amount)**.

- From **Locked**, a correct *enterPin* moves the lock to **Unlocked**; three consecutive wrong pins move it to **Jammed** (print an intrusion-alert message).
- From **Unlocked**, *lockButton()* returns the lock to **Locked**.
- From **Jammed**, only *forceUnlock(masterKey)* (with the correct key) can move the lock back to **Locked**; any other operation should print that the lock is jammed and do nothing.
- Whenever *drainBattery(amount)* causes the remaining battery to fall below 15%, the lock must transition to **LowBattery** regardless of its current state (except Jammed), and in this state only *forceUnlock* works; all other operations should print a warning to recharge.
- The design must make it easy to add a new state (e.g. **Maintenance**) later without modifying existing state classes.

Task: Implement this behaviour with an appropriate design pattern and demonstrate at least one full run touching every state.

Subsection: S2 Time: 25 minutes

Pattern focus: State Pattern + Observer Pattern

You are building the backend for a **ride-sharing app**. Every ride goes through the states **Requested** -> **Accepted** -> **OnTrip** -> **Completed**, with a possible **Cancelled** branch from **Requested** or **Accepted** only (a ride can no longer be cancelled once it is **OnTrip**).

- Each state enforces its own valid transitions; invalid ones (e.g. Completed -> OnTrip) should be rejected with a clear message and no state change.
- Independently of the state logic, both the **Rider** and the **Driver** must be automatically notified every time the ride's state changes, without the Ride class knowing anything about how the Rider/Driver display their notifications.
- It should be possible to add a third observer later (e.g. a **SupportTeam** dashboard) without touching the Ride class.

Task: Model the ride's lifecycle with one pattern and the notification mechanism with a second, complementary pattern. Demonstrate a full ride from request to completion, and a separate ride that gets cancelled, printing all notifications along the way.

Subsection: ST1 Time: 20 minutes

Pattern focus: Strategy Pattern

A logistics company wants to compute **shipping cost** for a package based on the delivery option chosen by the customer at checkout: **Standard** (flat rate + BDT 5/km), **Express** (flat rate double of Standard + BDT 12/km), and **SameDay** (flat rate triple of Standard + BDT 20/km, only available for distance ≤ 30 km).

- The customer selects a delivery option, and the system must use the corresponding strategy to compute the final cost for a given distance and weight.
- The company plans to add a **Bulk** shipping option in the future; adding it should not require modifying the checkout code.
- If SameDay is selected but the distance exceeds 30 km, the strategy should print a rejection message instead of a cost.

Task: Implement the shipping-cost calculation using the Strategy pattern and demonstrate all three options with at least one case that gets rejected.

Subsection: ST2 Time: 20 minutes

Pattern focus: Strategy Pattern

You are building a text editor plugin that can **compress** a document before saving, using one of three algorithms chosen by the user: **ZipCompression**, **RarCompression**, or **NoCompression** (stores the file as-is).

- Users can change the compression strategy at any time, even for documents that were previously saved with a different strategy.
- Each strategy must independently report how many bytes were saved compared to the original size (you may simulate this with simple arithmetic, no real compression needed).
- New algorithms should be pluggable without changing the SaveManager class.

Task: Implement this using the Strategy pattern. Demonstrate saving the same document three times, once per strategy, printing the size saved each time, and show switching the strategy at runtime.

Subsection: O1 Time: 20 minutes

Pattern focus: Observer Pattern

Design a **Weather Station** that reads temperature and humidity and pushes updates to any number of registered displays: a **CurrentConditionsDisplay** (prints temperature and humidity), a **StatisticsDisplay** (tracks and prints min/max/average temperature seen so far), and a **AlertDisplay** (prints a heat-warning only when temperature exceeds 40 degrees).

- The WeatherStation should not know the concrete types of its displays, only that they can be notified.
- Displays can be registered and unregistered at runtime (e.g. AlertDisplay unsubscribes once summer ends).
- It should be trivial to add a new kind of display later without changing WeatherStation.

Task: Implement this using the Observer pattern. Simulate at least 4 weather readings, with one display unsubscribing partway through, and show that it stops receiving updates afterward.

Subsection: O2 Time: 25 minutes

Pattern focus: Observer Pattern + Strategy Pattern

A crypto exchange wants a **PriceAlertBot**. Multiple users can subscribe to a coin's price feed (e.g. BTC, ETH). Whenever a coin's price changes, every subscribed user should be evaluated for whether their personal alert condition has been triggered.

- Two users may use different alert conditions on the same coin: an **AboveThreshold** strategy (fires when price goes above a value) and a **PercentDropStrategy** (fires when price drops by more than X% from the last update).
- The CoinFeed class must not know which alert condition each user is using -- only that it can ask each subscriber to evaluate the new price.
- New alert conditions (e.g. BelowThreshold) should be addable without modifying CoinFeed or existing user code.
- Users can subscribe/unsubscribe from a coin at runtime.

Task: Use the Observer pattern for the subscription/notification mechanism and the Strategy pattern for each user's alert condition. Demonstrate at least 2 users on the same coin with different alert strategies, across at least 3 price changes, with one user unsubscribing partway through.

Subsection: M1 Time: 20 minutes

Pattern focus: Mediator Pattern

You are building a simplified **Air Traffic Control** simulator. Several **Flight** objects (e.g. Flight101, Flight202, Flight303) need to request permission to land or take off from a single runway, but flights must never talk to each other directly.

- Each Flight sends its request (*requestLanding* or *requestTakeoff*) only to a central **ControlTower**.
- The ControlTower decides: if the runway is free, it grants the request and marks the runway as busy; otherwise it tells the requesting flight to hold and queues the request.
- When a flight finishes landing/take-off, it notifies the ControlTower, which frees the runway and immediately grants the next queued request, if any.

Task: Implement this with the Mediator pattern and demonstrate at least 3 flights making requests, including at least one that must wait because the runway is busy.

Subsection: M2 Time: 25 minutes

Pattern focus: Mediator Pattern + State Pattern

Design the control logic for a small building with two **Elevator** cars, coordinated by a single **ElevatorController** (mediator) so the cars never communicate directly with each other.

- Each Elevator has its own state -- **Idle**, **MovingUp**, **MovingDown**, and **DoorsOpen** -- and only allows valid transitions (e.g. it cannot go directly from DoorsOpen to MovingUp without first returning to Idle).
- When a floor button is pressed, the request goes to the ElevatorController, which decides which elevator should respond (e.g. whichever is Idle and closer to the requested floor) and instructs that elevator to move.
- If both elevators are busy, the controller should queue the request and dispatch it once an elevator becomes Idle.

Task: Model each elevator's behaviour with the State pattern and the dispatch logic with the Mediator pattern. Demonstrate at least 3 floor requests, with at least one that must be queued.

Subsection: T1 Time: 20 minutes

Pattern focus: Template Method Pattern

You are building a **report export** feature. Every export, regardless of format, follows the same overall sequence: (1) gather data, (2) apply header, (3) write body, (4) apply footer, (5) finalize file -- but each concrete format customizes some of these steps.

- **PdfReportExporter**: header prints a styled title block; body prints rows with a page-break simulation every 3 rows; footer prints a page-number-style note.
- **CsvReportExporter**: header prints a comma-separated column line; body prints comma-separated rows; footer is skipped entirely (CSV has no footer -- make this step optional in the base class).
- The gather-data and finalize steps are identical for every format and must live in exactly one place.

Task: Implement this using the Template Method pattern and run the export for both formats on the same sample data, showing the differing output clearly.

Subsection: T2 Time: 25 minutes

Pattern focus: Template Method Pattern + Strategy Pattern

You are building a **data-import pipeline** that always performs the same four steps in order: (1) read raw records from a source, (2) validate each record, (3) sort the valid records, (4) print an import summary. Two source types are supported: **CsvSource** and **JsonSource**, which differ only in how step (1) parses raw text into records and how step (2) validates a record (CSV requires exactly 3 fields per record; JSON requires a non-null 'id' field).

- Steps (1) and (2) must be overridden per source type; steps (3) and (4) are shared by all sources and must not be duplicated.
- Step (3), the sorting step, must be pluggable independently of the source type: the caller can choose **SortByIdStrategy** or **SortByNameStrategy** at runtime, and this choice must work identically for both **CsvSource** and **JsonSource**.
- Invalid records found in step (2) should be skipped and counted, then reported in the final summary.

Task: Implement the fixed pipeline skeleton with the Template Method pattern and the interchangeable sorting step with the Strategy pattern. Demonstrate running the pipeline for both source types, each with a different sort strategy, on sample data that includes at least one invalid record.